C Programming — Core Programs & Concepts

VTU Semester-End Examination Notes — Modules 2 to 5

This document covers six frequently examined VTU programming questions spanning Modules 2 to 5: control statements (switch-case), array-based programs (matrix multiplication, bubble sort, string handling), and pointers & structures. Each topic includes the concept, a diagram where useful, complete working C code, and a sample output / dry run.

#	Topic	Module
1	switch-case statement, with example	M2
2	Matrix multiplication program	M3
3	Bubble sort program	M4
4	String concatenation without built-in function	M4
5	Pointer declaration/initialization + sum, mean, std. deviation program	M5
6	Structure declaration with student example	M5

1. switch-case Statement, with Example

Module 2 — Decision Control Statements

The **switch** statement is a multi-way decision-making construct that tests whether an expression matches one of several constant integer/character values (**case** labels), and branches to the matching case. It is often a cleaner alternative to a long chain of `else-if` statements.

Syntax

```
switch (expression)
{
  case value1:
    statement(s);
    break;
  case value2:
    statement(s);
    break;
    ...
  default:
    statement(s);
}
```

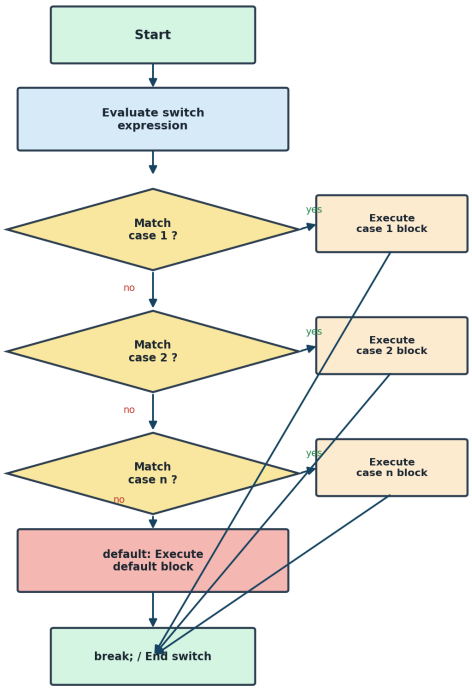


Fig. 1 — Flowchart of the switch-case statement

Rules

- The switch expression must evaluate to an integer or character value (not float/double or string).
- Each `case` label must be a constant and must be unique.
- The `break` statement is used to exit the switch after a matching case executes — without it, execution continues ("falls through") into the next case.
- The `default` case is optional and executes when no case matches.

Example Program — simple calculator using switch-case

```c
#include <stdio.h>
int main()
{
  int a, b;
  char op;

  printf("Enter two numbers: ");
  scanf("%d %d", &a, &b);
  printf("Enter operator (+,-,*,/): ");
  scanf(" %c", &op);

  switch (op)
  {
    case '+':
      printf("Result = %d\n", a + b);
      break;
    case '-':
      printf("Result = %d\n", a - b);
      break;
    case '*':
      printf("Result = %d\n", a * b);
      break;
    case '/':
      if (b != 0)
        printf("Result = %.2f\n", (float)a / b);
      else
        printf("Division by zero!\n");
      break;
    default:
      printf("Invalid operator!\n");
  }
  return 0;
}
```

Sample Output: Enter two numbers: 10 5 Enter operator (+,-,*,/): * → Result = 50

2. Matrix Multiplication Program

Module 3 — Arrays

Two matrices A (of order m×n) and B (of order n×p) can be multiplied only when the number of columns of A equals the number of rows of B. The resulting matrix C has order m×p, where each element C[i][j] is the sum of the products of the corresponding row of A and column of B:

$C[i][j] = \Sigma\ (A[i][k] \times B[k][j])$ for k = 0 to n-1

C Program

```c
#include <stdio.h>
int main()
{
  int a[10][10], b[10][10], c[10][10];
  int i, j, k, m, n, p, q;

  printf("Enter rows & columns of matrix A: ");
  scanf("%d %d", &m, &n);
  printf("Enter rows & columns of matrix B: ");
  scanf("%d %d", &p, &q);

  if (n != p)
  {
    printf("Matrix multiplication not possible!\n");
    return 0;
  }

  printf("Enter elements of matrix A:\n");
  for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
      scanf("%d", &a[i][j]);

  printf("Enter elements of matrix B:\n");
  for (i = 0; i < p; i++)
    for (j = 0; j < q; j++)
      scanf("%d", &b[i][j]);

  /* multiply the two matrices */
  for (i = 0; i < m; i++)
  {
    for (j = 0; j < q; j++)
    {
      c[i][j] = 0;
      for (k = 0; k < n; k++)
        c[i][j] += a[i][k] * b[k][j];
    }
  }

  printf("Product matrix C:\n");
  for (i = 0; i < m; i++)
  {
    for (j = 0; j < q; j++)
      printf("%d \t", c[i][j]);
    printf("\n");
  }
  return 0;
```

```
}
```

Dry Run Example

A (2×2) = [[1,2],[3,4]] B (2×2) = [[5,6],[7,8]]

C[0][0] = 1×5 + 2×7 = 19 C[0][1] = 1×6 + 2×8 = 22

C[1][0] = 3×5 + 4×7 = 43 C[1][1] = 3×6 + 4×8 = 50

Result C = [[19, 22], [43, 50]]

3. Bubble Sort Program

Module 4 — Arrays & Sorting

Bubble sort is a simple sorting algorithm that repeatedly steps through the array, compares each pair of adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the whole array is sorted — with each full pass, the largest remaining unsorted element "bubbles up" to its correct position at the end.

C Program

```c
#include <stdio.h>
int main()
{
  int arr[50], n, i, j, temp;

  printf("Enter number of elements: ");
  scanf("%d", &n);

  printf("Enter %d elements: ", n);
  for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

  /* bubble sort logic */
  for (i = 0; i < n - 1; i++)
  {
    for (j = 0; j < n - i - 1; j++)
    {
      if (arr[j] > arr[j + 1])
      {
        temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }

  printf("Sorted array: ");
  for (i = 0; i < n; i++)
    printf("%d \t", arr[i]);
  return 0;
}
```

Dry Run Example — sorting {5, 1, 4, 2, 8}

Pass	Array after pass	Comparisons made
Original	5, 1, 4, 2, 8	—
Pass 1	1, 4, 2, 5, 8	(5,1)→swap, (5,4)→swap, (5,2)→swap, (5,8)→no swap
Pass 2	1, 2, 4, 5, 8	(1,4)→no, (4,2)→swap, (4,5)→no
Pass 3	1, 2, 4, 5, 8	(1,2)→no, (2,4)→no
Pass 4	1, 2, 4, 5, 8	(1,2)→no — array is sorted

Time complexity: $O(n^2)$ in the worst and average case, O(n) in the best case (already sorted, when an optimization flag is used). **Space complexity:** O(1) — sorting is done in-place.

4. String Concatenation Without a Built-in Function

Module 4 — Strings

String concatenation means appending the characters of one string to the end of another. Normally the library function `strcat()` from `string.h` is used, but it can also be done manually by copying characters one at a time using array indexing — without calling any built-in string function.

Logic

1. Find the length (index of the null character '\0') of the first string, str1.
2. Starting from that index in str1, copy each character of the second string, str2, one by one.
3. After copying all characters of str2, terminate str1 with a null character '\0'.

C Program

```c
#include <stdio.h>
int main()
{
  char str1[100], str2[50];
  int i = 0, j = 0;

  printf("Enter first string: ");
  scanf("%s", str1);
  printf("Enter second string: ");
  scanf("%s", str2);

  /* find length of str1 (locate the null character) */
  while (str1[i] != '\0')
    i++;

  /* copy each character of str2 onto the end of str1 */
  while (str2[j] != '\0')
  {
    str1[i] = str2[j];
    i++;
    j++;
  }

  str1[i] = '\0'; /* terminate the concatenated string */

  printf("Concatenated string: %s\n", str1);
  return 0;
}
```

Sample Output

```
Enter first string: Good
Enter second string: Morning
Concatenated string: GoodMorning
```

5. Pointer Declaration/Initialization & Sum, Mean, Std. Deviation

Module 5 — Pointers

A **pointer** is a variable that stores the memory address of another variable, rather than a data value directly. Pointers are declared using the * operator and are initialized using the address-of operator &.

Declaration and Initialization

```
int x = 10;
int *p; /* declaration: p is a pointer to an int */
p = &x; /* initialization: p now holds the address of x */

/* declaration + initialization together: */
int *p = &x;
```

Here, &x gives the **address** of x, and *p (the dereference / indirection operator) gives the **value** stored at the address that p is pointing to — so *p is equivalent to x itself (value 10).

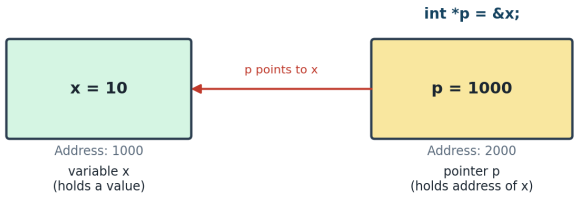


Fig. 2 — A pointer variable p holding the address of variable x

Statistical formulas used: for n values x_1, x_2, ..., x_n
Mean (μ) = (Σx_i) / n
Standard deviation (σ) = $\sqrt{(\Sigma(x_i - \mu)^2 / n)}$

C Program — sum, mean and standard deviation using pointers

```
#include <stdio.h>
#include <math.h>
int main()
{
  int n, i;
  float arr[50], sum = 0, mean, variance = 0, std_dev;
  float *p; /* pointer to float */

  printf("Enter number of values: ");
  scanf("%d", &n);

  p = arr; /* p now points to the base of array arr */
  printf("Enter %d values: ", n);
  for (i = 0; i < n; i++)
    scanf("%f", (p + i)); /* store via pointer */

  /* calculate sum using pointer arithmetic */
  for (i = 0; i < n; i++)
    sum = sum + *(p + i);
```

```
  mean = sum / n;

  /* calculate variance */
  for (i = 0; i < n; i++)
    variance = variance + (*(p + i) - mean) * (*(p + i) - mean);
  variance = variance / n;
  std_dev = sqrt(variance);

  printf("Sum = %.2f\n", sum);
  printf("Mean = %.2f\n", mean);
  printf("Standard Deviation = %.2f\n", std_dev);
  return 0;
}
```

Sample Output (for values 2, 4, 4, 4, 5, 5, 7, 9): Sum = 40.00, Mean = 5.00, Standard Deviation = 2.00. Compile with `gcc prog.c -o prog -lm` (the `-lm` flag links the math library for `sqrt()`).

6. Structure Declaration — Student Example

Module 5 — Structures

A **structure** is a user-defined data type in C that groups together variables of different data types under a single name, so that related information can be treated as one unit. Unlike an array, whose elements must all be of the same type, a structure's members can each have a different type.

Syntax of Structure Declaration

```
struct tag_name
{
  data_type member1;
  data_type member2;
  ...
};
```

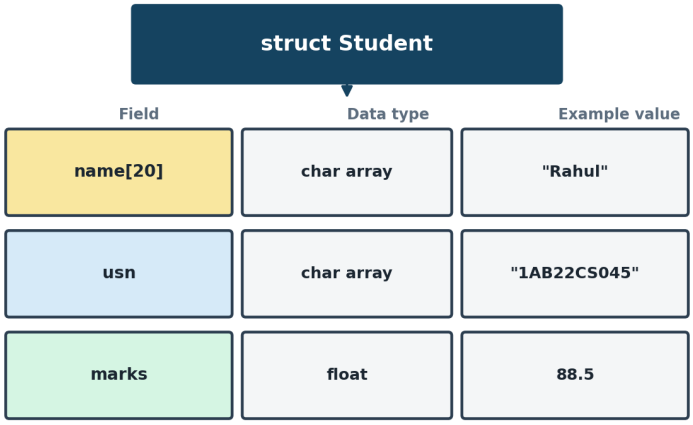


Fig. 3 — struct Student and its members

A structure variable is declared either along with the structure definition, or later using the `struct tag_name variable_name;` syntax. Members are accessed using the **dot (.) operator**: `variable_name.member_name`.

C Program — store and display student details using a structure

```
#include <stdio.h>

struct Student /* structure declaration */
{
  char name[20];
  char usn[15];
  float marks;
};

int main()
{
  struct Student s1; /* structure variable */

  printf("Enter name: ");
  scanf("%s", s1.name);
  printf("Enter USN: ");
  scanf("%s", s1.usn);
```

```
  printf("Enter marks: ");
  scanf("%f", &s1.marks);

  printf("\n--- Student Details ---\n");
  printf("Name : %s\n", s1.name);
  printf("USN : %s\n", s1.usn);
  printf("Marks : %.2f\n", s1.marks);
  return 0;
}
```

Sample Output

```
Enter name: Rahul
Enter USN: 1AB22CS045
Enter marks: 88.5

--- Student Details ---
Name : Rahul
USN : 1AB22CS045
Marks : 88.50
```

Note: An array of structures — e.g. `struct Student class[50];` — can be used to store records for many students, with each record accessed as `class[i].name`, `class[i].marks`, etc.